CS_5080_Lecture_16

Tuesday, April 14, 2026 4:48 PM

This lecture seems to be mainly a review part of the

Policy in RL

$\pi(s) = a, \forall s \in S$ deterministic policies

$s \in S$ probability of 1

$a \in A$

Non-deterministic policy

probability not fixed

$\pi(a|s) = \text{prob}(A_t = a | S_t = s)$

looking at page 80 in the book policy iteration Chap 4.3

learning policy via learning values

looking at chap.5 Monte carlo in book Page 99 Monte Carlo E

1) learned $v(s)$ and then used dynamics with v values to learn policy

looking at page 103

2) learn $Q(s,a)$ values → learn policy

learn a policy by learned values

looking at chap 6 pg 130 Sarsa TD algorithms pg 131 Q-learning - no policy at all

3) learn values $Q(s,a)$ values no policy learning

* schools of thought *

1) Learn policy via values
2) learn values only
3) learn policy directly without values
4) learn policy with the help of values
5) learn policy via LLM - cheat? easy way out? Lol

Policy is easier to learn than values? Some papers mention this theory - learning policy is better... need empirical data

non-det - many claim only policy learning can do it. but, if we have learned Q values $\pi(s,a) = \frac{e^{q(s,a)}}{\sum_{a \in A} e^{q(s,a')}}$

* Start new material * Chap 13 Policy Gradient Methods

learn policy directly

initialize π - randomly or to all very small values

→ iterate over some experience - episodes
  ↳ update π
  Repeat

Updating of π

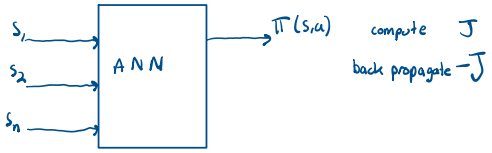
gradient, partial derivatives

$$\pi_{new} \leftarrow \pi_{old} + \alpha * \nabla J(\theta)$$

α — learning rate

J - function that measures goodness of the policy

θ - a vector of parameters



1) can we use a goodness function to train an ANN instead of loss or badness function.
2) what kind of data will replace tabular labeled data in supervised learning
3) how should the update function look like
4) what are possible goodness functions? measure/ functions

Regular ANN is supervised learning

data set with labeled features or random values y
feed to ANN
result comes out $\hat{y}$ - predicted value
loss function $\mathcal{L}(y,\hat{y})$ → measure of badness

Gradient descent

what is GD used in supervised ML to train ANNs

Assume an ANN with 1 parameter θ

Feature x	Label y
1	5
2	3.2
0	10
7	4
...	...
n	n_2

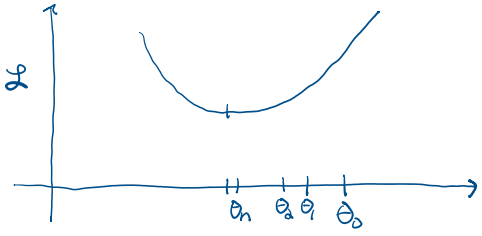
$x \xrightarrow{\theta} y$ → value from table = actual value

$\hat{y}$ → value from ANN = predicted value

$\mathcal{L}(y,\hat{y}) = \frac{1}{2}(y-\hat{y})^2$ squared error loss

[claude.ai] looks like we are 5610 territory here

Assume the loss function is convex



① θ_0 - initialize / Guess a solution
$i \to 0$

Repeat

update $\theta_{(i+1)}$, $\theta \leftarrow \theta + (\text{update to } \theta)$

until no improvement possible
or stop at a predetermined point

$\frac{1}{2}(y-\hat{f}(x;\theta))^2$ $\quad \hat{f}(x;\theta) = \hat{y}$

Stochastic Gradient Descent

② $\theta_0 \leftarrow$ init

$i \leftarrow 0$

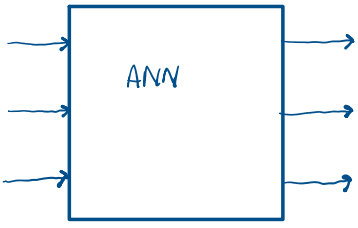
Repeat

$\theta_{i+1} \leftarrow \theta_i + \alpha\left(-\frac{d\mathcal{L}}{d\theta}\right)$ move a tiny bit

$-\nabla \mathcal{L}$ $\quad \mathcal{L}$ = loss function

i++

until no improvement or N iterations



$\vec{\theta} = \begin{bmatrix}\theta_0\\ \theta_1\\ \vdots\\ \theta_n\end{bmatrix}$

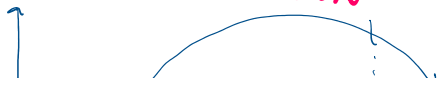
③ $\vec{\theta}$ — init all params randly

Repeat

$$\begin{bmatrix}\theta_0\\ \theta_1\\ \vdots\\ \theta_n\end{bmatrix} \leftarrow \begin{bmatrix}\theta_0\\ \theta_1\\ \vdots\\ \theta_n\end{bmatrix} - \alpha \begin{bmatrix}\partial\mathcal{L}/d\theta_0\\ d\mathcal{L}/d\theta_1\\ \vdots\\ d\mathcal{L}/d\theta_n\end{bmatrix}$$

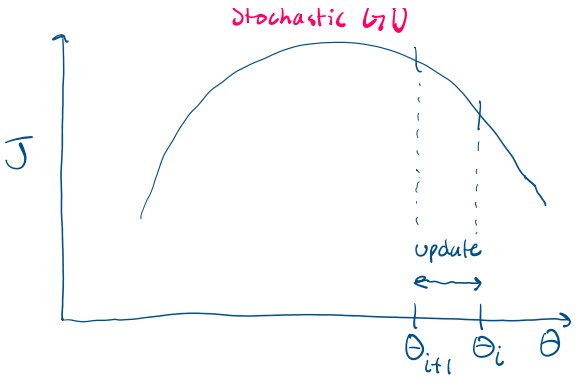
until....

Stochastic GD



④ $\vec{\theta}$ — initialize

Repeat



④ $\vec{\theta}$ – initialize

Repeat

$\vec{\theta} \leftarrow \vec{\theta} - \alpha \nabla \mathcal{L}\big|_{\vec{x}}$

until....

⑤ RL update

$\vec{\theta} \leftarrow$ init

repeat

$\vec{\theta} \leftarrow \vec{\theta} - \alpha(-\nabla J)\big|_{\text{experience}}$

until.....